Visual Question Answering and the relation between language, perception and the world

Staffan Larsson, Bill Noble and Robin Cooper
University of Gothenburg, Sweden

NALOMA, August 4, 2026

Outline

Outline

Introduction I

- We outline a general account of VQA using a perception-oriented formal semantic framework.
- VQA can be viewed as a linguistically and philosophically relevant task that can help us better understand the relation between language, perception and the world.
- We argue that *linguistic meaning helps us structure our takes on visual scenes, enabling us to classify perceived situations* so that we e.g. can answer questions and determine whether a sentence correctly describes a scene.
- When studying abstract matters like the relation between language, perception and the world, it is often helpful to look at specific problems.

VQA I

- Visual Question Answering (VQA) is a machine learning task where the model is provided with an image and a natural language question and must provide an answer to the question based on the image.
- The task is focused on the natural language and visual understanding capabilities of the model.
- Many VQA datasets are susceptible to relatively superficial correlations between the inputs and the answer, which allow machine learning models to avoid the complexity involved in associating language and world more generally.

LLMs

- While Visual LLMs generalize better on VQA and related tasks when compared to older vision-and-language models, LLMs are essentially "black boxes" which can be trained and deployed but whose inner workings are opaque.
- Hence, LLMs do not necessarily help us derive insights into the relation between language, perception and the world.

Formal semantics I

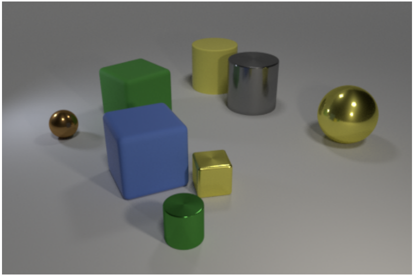
- In formal semantics in the Montague tradition and Possible World Semantics, the meaning of a word such as “dog” is taken to be a function from possible worlds (and times) to its extension, i.e. the set of all dogs in that world.
- Such logical representations, however, have no natural interpretation in terms of neural implementation (be it artificial or biological), and offer no biologically plausible explanation of how language is related to sensory perception and the physical world.

Formal semantics II

- In modern computational linguistics the meaning of a word such as "dog" might be related to a statistically-based perceptual classifier that is able to recognize dogs
- This corresponds to our ability to recognize a dog when we see one and is also much more hopeful in terms of neural modelling.
- We adopt a view of linguistic meaning that relates to the notion of classifiers while at the same time preserving important techniques from formal semantics, such as the treatment of compositionality.

CLEVR

- In this paper, we outline a general account of VQA
- We take the CLEVR dataset (Johnson *et al.*, 2016) as an example and using a perception-oriented formal semantic framework.[1]
- What is involved in answering the question about the scene?
- Q: *What color is the cylinder in front of the yellow cube?*



[1] https://github.com/GU-CLASP/types-vectors-spiking-neurons/

Outline

TTR fundamentals I

- $a : T$ is a judgment that a is of type T
- Types may be either *basic* or *complex*
- Some basic types in TTR:
  - *Ind*, the type of an individual
  - *Int*, the type of integers
  - *Real*, the type of real numbers

TTR fundamentals II

- Complex types are structured objects which have types or other objects introduced in the theory as components
- *ptypes* are constructed from a predicate and arguments of appropriate types as specified for the predicate.
- Examples are 'man(a)', 'see(a,b)' where $a, b : Ind$.

TTR fundamentals III

- Wh-questions are lambda abstracts over ptypes
  - "The dog chases the cat": chase(dog_{52}, cat_{43})
  - "Who chases the cat?": λx.chase(x, cat_{43})
  - "Who does the dog chase?": λx.chase(dog_{52}, x)

TTR fundamentals IV

- A sample record and record type:

$$\begin{bmatrix} \text{ref} & = & \text{obj}_{123} \\ \text{c}_{\text{man}} & = & \text{prf1} \\ \text{c}_{\text{run}} & = & \text{prf2} \end{bmatrix} : \begin{bmatrix} \text{ref} & : & \textit{Ind} \\ \text{c}_{\text{man}} & : & \text{man(ref)} \\ \text{c}_{\text{run}} & : & \text{run(ref)} \end{bmatrix}$$

- The record on the left is of the record type on the right provided
  - $\text{obj}_{123} : \textit{Ind}$
  - $\text{prf1} : \text{man}(\text{obj}_{123})$
  - $\text{prf2} : \text{run}(\text{obj}_{123})$
- Types constructed with predicates may be *dependent*.
- This is represented by the fact that arguments to the predicate may be represented by labels used on the left of the ':' elsewhere in the record type.
- Above, the type of c_{man} is dependent on ref (as is c_{run}).
- Note that a dependent ptype such as run(ref) actually shorthand for $\langle \lambda x : \textit{Ind}.\text{run}(x), \langle \text{ref} \rangle \rangle$

TTR fundamentals V

- If r is a record and ℓ is a label in r, we can use a *path* $r.\ell$ to refer to the value of ℓ in r.
- Similarly, if T is a record type and ℓ is a label in T, $T.\ell$ refers to the type of ℓ in T.
- Records (and record types) can be nested, so that the value of a label is itself a record (or record type).

TTR fundamentals VI

- A fundamental type-theoretical intuition is that something of a ptype T is whatever it is that counts as a proof of T.
- One way of putting this is that "propositions are types of proofs".
- Above, we simply used prf$n$ as a placeholder for proofs; below, we will show how low-level perceptual input can be included in proofs.

TTR fundamentals VII

- Judgements (and other type actions, see Cooper (2023)) are regulated by action rules of the form

$$\frac{\varphi_0, \varphi_1, \ldots, \varphi_n}{\psi}$$

- Each φ_i is either an action or some other condition that must hold true and ψ must be a type action such as a judgement.
- The wavy line as opposed to a straight line indicates that this is not a rule of inference but that the premises above the line holding *license* or *afford* the action below the line.
- Thus there is no guarantee that the agent will carry out the action even if all the premises hold.

Outline

Related work I

- VQA is a rich test bed for investigating the compositional nature of– and relationships between language, perception, and reasoning.
- Because of its structure, the CLEVR dataset has been used extensively to test systems that explicitly combine these components.
- 1. Work exploiting the ground-truth question semantics of CLEVR itself, where the meaning of a question is represented as a procedure to determine its answer: Hudson and Manning (2018), Johnson *et al.* (2017), Yi *et al.* (2018)
- 2. Recent efforts to combine neural network representations with linguistic formal semantics:
  - Using a Convolutional Neural Network (CNN) to extract entity representations from images (Liu and Emerson, 2022) – inspired by linguistic theory
  - Using neural network-based entity representations embedded in a dense vector space as the basis for interpreting types in a dependent type system (Bekki *et al.*, 2021)

Related work II

- 3. Work on VQA involving TTR
  - In Dobnik and Cooper (2017), data is assumed to be a point map representing a visual scene.
  - A function takes a point map and produces a set of record types predicating properties of individuals
  - However, only the types of such records are provided, not the records themselves.
  - An implemented variant of the account in Dobnik and Cooper (2017) uses YOLO classifiers applied to image data and limited to polar questions (Matsson *et al.*, 2019)
  - The account in Utescher (2019) is more similar in spirit to the approach suggested in this paper, but does not use linguistic meaning to structure perceptual data

Related work III

- Existing work leaves open the question of what constitutes a witness for a ptype reflecting the meaning of an utterance referring to a visual scene
- We want to suggest an answer this question: Witnesses of such ptypes consists of tuples of high-level perceptual data (H-data) associated with the individuals involved in the ptype (typically, the arguments of a predicate).

Related work IV

- We also show how, when relating linguistic expressions to visual scenes, we *use linguistic meaning to structure our take on the visual scene*
- This includes finding the referents of linguistic expressions referring to objects in the scene.
- Furthermore, the account presented here is meant to generalise over specific kinds of visual data and classifiers.

Outline

A general TTR account of VQA

- In earlier work (see paper for references) we present work towards a general formal perception-oriented semantics.
- Here, we revise and extend this account to fit with the VQA task.
- We then show how the general account can be adapted to the CLEVR dataset, where H-data is regarded as CLEVR scene graphs.

Outline

Low-level and high-level perceptual data I

- When talking about an agent's perception of a sitation, we can call the "raw" input "low level perceptual data" (L-data).
- This can be e.g. the retinal image in a biological system or a digital image from a camera connected to a computer, or a point map
- When processing perceptual data, L-data is turned into "high level perceptual data" (H-data).
- H-data is the result of processing L-data, and this processing may include e.g. object detection, individuation, feature detection and compression.
  - Individuation is needed to pick out the part of L-data that concerns a single individual.
- A biological example of H-data is visual cortical information (simplifying somewhat, the output of the visual cortex when fed the retinal image as input).

Low-level and high-level perceptual data II

- We assume that H-data is individuated, which in TTR terms means that sections of H-data are associated with different objects of type *Ind*
- We define a type *HScene* as a list of H-data items corresponding to individuals in a perceived scene.

$$HScene{=}List(\left[\begin{array}{lll} \text{id} & : & Ind \\ \text{data} & : & HData \end{array}\right])$$

- We can now define a scene-specific judgement of being of a type *Ind* in an H-scene h:

$$\text{For } h{:}HScene,\ a :_h Ind \text{ iff for some } n,\ a = h.n.\text{id}$$

- We here take a list to be a record labelled by natural numbers, so $\left[\begin{array}{lll} 1 & = & v_1 \\ \ldots & & \\ \text{n} & = & v_n \end{array}\right]$):List($T$) provided $v_1 : T, \ldots, v_n : T$

Low-level and high-level perceptual data III

- We use a hash table to access H-data associated with an individual (i.e., an object of type *Ind*)

  If $h : HScene$ and $a :_h Ind$ then $\#_h a = h.n.data$ for n such that $a = h.n.id$

- We will surpress the subscript $_h$ and just write $\#a$
- The scope of the H-data associated with an individual a is basically "any available information that can be relevant to understanding an utterance referring to a".

Outline

Perceptual data as evidence for ptypes I

- In general, we will be using H-data as evidence/proofs for ptypes, echoing the dictum "propositions are types of proofs".
- Ptypes constructed from an n-place predicates with arguments $a_1, \ldots, a_n$ have an n-tuple $\langle \# a_1, \ldots, \# a_n \rangle$ as evidence.
  - $\langle \#\text{a} \rangle$: yellow(a)
  - $\langle \#\text{a} \rangle$: cube(a)
  - $\langle \#\text{b}, \#\text{a} \rangle$: in-front(b,a)
  - $\langle \#\text{b} \rangle$: cylinder(b)

Perceptual data as evidence for ptypes II

- Judging whether some H-data evidence is of a ptype can be done using classifiers
- For example, for a colour classifier κ_{col} whose domain is *Hdata* and whose range are colour labels RED, BLUE etc, and assuming that a and b are variables over objects of type *Ind*:
  - $\langle \#a \rangle$: red(a) if $\kappa_{\text{col}}(\#a)$=RED
  - $\langle \#a \rangle$: blue(a) if $\kappa_{\text{col}}(\#a)$=BLUE

    ...
- ...and similarly for 2-place ptypes assuming a left-or-right classifier κ_{lor}:
  - $\langle \#a, \#b \rangle$: right(a, b) if $\kappa_{\text{lor}}(\#a, \#b)$=RIGHT
  - $\langle \#a, \#b \rangle$: left(a, b) if $\kappa_{\text{lor}}(\#a, \#b)$=LEFT

Outline

Natural language semantics I

- In Larsson (2020), meanings of linguistic expressions are defined as objects of type *Mng*:

$$Mng = \begin{bmatrix} \mathrm{bg} & : & \mathit{RecType} \\ \mathrm{intrp} & : & \mathrm{bg} \rightarrow \mathit{Type} \\ \mathrm{clfr} & : & \mathrm{bg} \rightarrow \mathit{Type} \end{bmatrix}$$

- Intuitively, for a declarative utterance with meaning $p{:}Mng$,
  - p.bg is the type of the context in p is appropriately uttered
  - p.intrp is the context-independent meaning of the utterance (a function from a situation to a type)
  - p.clfr is a function that returns p if the utterance of p holds (is true) of the (take on the) situation.

Natural language semantics II

- In general, the bg field can be understood as capturing the presupposition of a linguistic expression.
- When interpreting linguistic expressions referring to visual scenes, an agent needs to structure their take on the scene to reflect these presuppositions.
- For example, "the red cube" needs to be mapped to a specific object (which is red and a cube) in the agent's take on the scene.
- This means that the agent, starting from low-level perceptual data, needs to individuate an object which is classified as both red and as cube

Natural language semantics III

- For an utterance u of a declarative sentence with meaning $p{:}Mng$ and a situation $s{:}p.\text{bg}$, we can provide an action rule specifying when an agent A may judge (A's take on the situation) s to be of the type specified by the (situated interpretation of) u in s.

$$\frac{p :_A Mng \qquad s :_A p.\text{bg} \qquad p.\text{clrf}(s) = p.\text{intrp}(s)}{s :_A p.\text{intrp}(s)}$$

- So an agent is licensed to judge s to be of the type $p.\text{intrp}(s)$ specified by the (situated interpretation of the) utterance u in s provided that s fulfills the presuppositions of p, and A perceives the situation to be of type $p.\text{intrp}(s)$.

Natural language semantics IV

- For example, the meaning of the sentence p="The cylinder in front of the yellow cube is green" could be:

$$
\mathrm{p}=\begin{bmatrix}
\mathrm{bg} & = & \begin{bmatrix}
\mathrm{x} & : & \mathit{Ind} \\
\mathrm{colour_x} & : & \mathrm{yellow(x)} \\
\mathrm{shape_x} & : & \mathrm{cube(x)} \\
\mathrm{y} & : & \mathit{Ind} \\
\mathrm{place_{yx}} & : & \mathrm{in\text{-}front(y,x)} \\
\mathrm{shape_y} & : & \mathrm{cylinder(y)}
\end{bmatrix} \\
\multicolumn{3}{l}{\mathrm{intrp}=\lambda r:\mathrm{bg}.\left[\mathrm{colour_y}:\mathrm{green}(r.\mathrm{y})\right]} \\
\multicolumn{3}{l}{\mathrm{clfr}\ =\lambda r:\mathrm{bg}.\ \left[\mathrm{colour_y}:T_{\mathrm{col}}(r.\mathrm{y})\right]}
\end{bmatrix}
$$

- . . . where T_{col} is a dependent type

Natural language semantics V

- More specifically, T_{col} is a dependent type that takes one or several individuals for which there is relevant H-data, calls a classifier function κ_{col} with the relevant H-data as input, and returns a ptype, e.g. red(r.y) or green(r.y) depending on the output of the classifier:

$$T_{\text{col}} : \mathit{Ind} \rightarrow \mathit{Type}$$

$$T_{\text{col}}(x) = \begin{cases} \text{red}(x) \text{ if } \kappa_{\text{col}}(\langle \# x \rangle) = \text{RED} \\ \text{blue}(x) \text{ if } \kappa_{\text{col}}(\langle \# x \rangle) = \text{BLUE} \\ \ldots \end{cases}$$

- For a two-place relation:

$$T_{\text{place}}(x, y) = \begin{cases} \text{left}(x,\ y) \text{ if } \kappa_{\text{lor}}(\langle \# x, \# y \rangle) = \text{LEFT} \\ \text{right}(x,\ y) \text{ if } \kappa_{\text{lor}}(\langle \# x, \# y \rangle) = \text{RIGHT} \end{cases}$$

Natural language semantics VI

- Assuming that s provides information that can be used by a classifier to decide whether the cylinder in front of the yellow cube is green or not, an agent A may judge

  $$s :_A \text{green}(s.\text{y})$$

  provided that

  $$T_{\text{col}}(s.\text{y}) = \text{green}(s.\text{y})$$

Natural language semantics VII

- As mentioned above, H-data tuples are regarded as objects of ptypes that witness (i.e. provide evidence or proof of) the ptype.
- Consequently, a record type containing ptypes (such as the bg field above) can be instantiated by a record where values of ptype fields are H-data tuples.
- We use such records to represent an agent's *take* on a scene, a so-called 'witness take' .
- A situated interpretation of a linguistic expression e is attained by applying $[e]$.bg to a witness take s.

Natural language semantics VIII

- Assume that the declarative sentence above is stated about a focus situation s_{123} to an agent A whose take on this situation is

$$\mathrm{s}^{A}_{123} = \begin{bmatrix} \mathrm{x} & = & \mathrm{a} \\ \mathrm{colour_x} & = & \langle \#\mathrm{a} \rangle \\ \mathrm{shape_x} & = & \langle \#\mathrm{a} \rangle \\ \mathrm{y} & = & \mathrm{b} \\ \mathrm{place_{yx}} & = & \langle \#\mathrm{b}, \#\mathrm{a} \rangle \\ \mathrm{shape_y} & = & \langle \#\mathrm{b} \rangle \\ \mathrm{colour_y} & = & \langle \#\mathrm{b} \rangle \end{bmatrix}$$

Natural language semantics IX

- To evaluate whether p is true for s^A_{123}, A computes p.intrp(s^A_{123}) and p.clfr(s^A_{123}) and checks for equality.
  - p.intrp(s^A_{123})= $\lambda r{:}\left[\begin{array}{ll}\text{x} & :\mathit{Ind}\\ \text{colour}_\text{x} & :\text{yellow(x)}\\ \text{shape}_\text{x} & :\text{cube(x)}\\ \text{y} & :\mathit{Ind}\\ \text{place}_\text{yx} & :\text{in-front(y,x)}\\ \text{shape}_\text{y} & :\text{cylinder(y)}\end{array}\right] . \left[\text{colour}_\text{y}{:}\text{green}(r.\text{y})\right](s^A_{123})=$ $\left[\text{colour}_\text{y}{:}\text{green(b)}\right]$
  - p.clfr(s^A_{123}) $= \left[\text{colour}_\text{y}{:}\text{red(b)}\right]$
  - ... assuming that $\kappa_\text{col}(\langle\#\text{b}\rangle)=\text{RED}$ and hence $T_\text{col}(\text{b})=\text{red(b)}$.
- Recall

$$\frac{p :_A \mathit{Mng} \qquad s :_A p.\text{bg} \qquad p.\text{clrf}(s) = p.\text{intrp}(s)}{s :_A p.\text{intrp}(s)}$$

- In this case, the equality fails and hence p is not judged to be true of s_{123} (since s_{123} is not judged to be of type p.intrp(s_{123}))

Outline

Questions I

- For questions, the situated interpretation is not a ptype but instead a function from an answer (in this case, a predicate function) to a ptype.
- A question "What color is the cylinder in front of the yellow cube?" could then have the meaning:

$$q=\left[\begin{array}{lcl} \mathrm{bg} & = & \left[\begin{array}{ll} \mathrm{x} & :\mathrm{Ind} \\ \mathrm{colour_x} & :\mathrm{yellow(x)} \\ \mathrm{shape_x} & :\mathrm{cube(x)} \\ \mathrm{y} & :\mathrm{Ind} \\ \mathrm{place_{yx}} & :\mathrm{in\text{-}front(y,x)} \\ \mathrm{shape_y} & :\mathrm{cylinder(y)} \end{array}\right] \\ \mathrm{intrp} & = & \lambda r:\mathrm{bg}.\lambda P:(\mathit{Ind}\rightarrow \mathit{Type}). \\ & & \left[\ \mathrm{colour_y}\ :\ P(r.\mathrm{y})\ \right] \\ \mathrm{clfr} & = & \lambda r:\mathrm{bg}.\ \left[\mathrm{colour_y}: T_{\mathrm{col}}(r.y)\right] \end{array}\right]$$

Questions II

- The interpretation function allows an agent to compute a Ptype from the question and an (elliptical) answer, which is useful in integrating an agent's verbal answer to the question
- The classifier function can be used to compute the answer to the question based on the agent's take on the situation referred to.
- The latter is useful for figuring out the answer (which could be used when providing a verbal answer).

Questions III

- Assume that a question "What color is the cylinder in front of the yellow cube?" with meaning q is asked about a focus situation s_{123}.
- This question presupposes that there is a (unique) yellow cube and a (unique) cylinder in front of it.
- An agent who is asked this question might use their take on the scene to find an answer.

Questions IV

- For a question q, we use the classifier function q.clfr to find an answer by applying it to the take on the scene that the question is about
- This requires the scene to be of the type specified by q.bg.

$$\mathrm{q.clfr} = \lambda r{:}\begin{bmatrix} \mathrm{x} & : \mathit{Ind} \\ \mathrm{c}_{\text{col-x}} & : \mathrm{yellow(x)} \\ \mathrm{c}_{\text{shape-x}} & : \mathrm{cube(x)} \\ \mathrm{y} & : \mathit{Ind} \\ \mathrm{c}_{\text{fob-yx}} & : \text{in-front(y,x)} \\ \mathrm{c}_{\text{shape-y}} & : \mathrm{cylinder(y)} \end{bmatrix}.\ \begin{bmatrix} \mathrm{colour_y} : T_{\mathrm{col}}(r.\mathrm{y}) \end{bmatrix}$$

Finding an answer to a question I

- We can imagine the question above being asked about a focus situation s_{123} to an agent A whose take on this situation is s^{A}_{123} as above.

$$\mathrm{q.clfr}(\mathrm{s}^{A}_{123}) = \left[\ \mathrm{colour}_{\mathrm{y}} \quad : \quad \mathrm{red(b)}\ \right]$$

- . . . again assuming that $\kappa_{\mathrm{col}}(\langle\#\mathrm{b}\rangle)=\textsc{red}$.

Finding an answer to a question II

- Given the above, for a question q we want to take an H-scene h and construct a witness take s_h:q.bg that q.clfr can be applied to to find the answer to q.
- We will use T_{bg} interchangeably for q.bg

Outline

Definition of witness takes I

- To build $s_h : T_{\text{bg}}$ from h, we label the individual objects in h so that the ptype classifier results (ptypes) come out as specified in T_{bg}.
- To do this, we can take all possible labellings of (H-data concerning) individuals in the H-data and then filter them out by applying classifiers when typechecking against the ptypes in T_{bg}.
- Assuming an scene $h : \mathit{HScene}$, we create records from h, enumerating all possible labellings (records) $\mathcal{S}$ of h that match T_{bg}.

Definition of witness takes II

- For a record type T and an H-scene h , there is a function $\# : \mathit{Ind} \to \mathit{Hdata}$ and a set of records $\mathcal{S}$ (the set of witness takes) such that for each $s \in \mathcal{S}$:
    1. for all ℓ such that $T.\ell : \mathit{Ind}$, there is an $a :_h \mathit{Ind}$ such that $s.\ell = a$
    2. for all ℓ such that $T.\ell = \langle f, L\rangle$ where
        - $L = \langle \ell_1, \ldots, \ell_n\rangle$,
        - $s.\ell = \langle \#(s.\ell_1), \ldots \#(s.\ell_n)\rangle$
    3. $s : T$
- Regarding 2, recall that a ptype such as run(ref) actually shorthand for $\langle \lambda x : \mathit{Ind}.\mathrm{run}(x), \langle \mathrm{ref}\rangle\rangle$

Definition of witness takes III

- Given the above, we can now tentatively provide an action rule licensing an agent A answering a question q : *Question* based on perceptual data H : *HScene*

$$\frac{q :_A \mathit{Mng} \qquad H :_A \mathit{HScene} \qquad s_H :_A q.\mathrm{bg}}{:_A \textsc{Answer}(A, q.\mathrm{clfr}(s_H))!}$$

Building witness takes from H-data I

- We define a function BUILDTAKES that uses tree search to build up list of situation takes respecting T (see paper for details)
- The algorithm itself is not particularly important and probably not cognitively plausible
- What is of note here is the relationship between the H-data and the candidate situation takes, which are being built-up and structured according to the interpretation of the question.
- To satisfy the pre-conditions of the question (i.e., to find a witness for q.bg and candidate argument to q.clfr), it is not enough to use generic perceptual input
- Instead, we must build up (or structure) a *take* on the situation based on high-level perceptual data and the question itself.
- Only then can the situation take be used to answer the question.

Outline

Adapting the general account to the CLEVR dataset

- Everything we have said above is independent of the type of H-data.
- In this section, we will show a concrete instance of this general framework, where H-data is a set of CLEVR scene graphs.
- This is intended as an illustration and proof-of-concept only, showing an example of how the general TTR account applies to a specific type of H-data and classification method.

Outline

The CLEVR dataset I

- CLEVR (Johnson *et al.*, 2016) is a synthetic dataset of programatically rendered 3d scenes of geometric objects and template-generated questions about them.
- Drawn from an extremely constrained visual and linguistic domain domain.
- Objects are drawn from limited number of geometric solids and have a finite range of colors, sizes, and textures.

The CLEVR dataset II

- Since the dataset is programatically generated, it is possible to produce “ground truth” semantics for questions and schematic descriptions
- Ground truth scene representations are presented as a *scene graph*, whose nodes are objects, annotated with attributes like color and shape (one-place predicates), and whose edges represent binary spatial relations between pairs of objects.
- Questions are represented as a functional programs that operate on those graphs.

Outline

Using CLEVR for H-data I

- In general, classification of visual scenes may involve e.g. processing of visual information in deep neural networks.
- However, in CLEVR the visual scenes are already annotated with formal structures that we will exploit when defining the classifiers required.
- In effect, we will treat the annotations as H-data, despite not actually being the result of processing of L-data.
- The classifiers themselves will be rather trivial, and they are meant only as an illustration of how the general account can be adapted to a specific dataset.

Using CLEVR for H-data II

- Assume we have as a scene graph g, consisting of a list of object descriptions that in turn consist of feature-value pairs.
- The original CLEVR data relies on the order of the items.
- To avoid this, we massage the data so that it includes a field `'id'` to avoid reliance on order.

Using CLEVR for H-data III

```
[{'id': 'h0',
  '3d_coords': [2.408, -2.868, 0.350],
  'color': 'green',
  'material': 'metal',
  'pixel_coords': [213, 257, 7.728],
  'rotation': 233.806,
  'shape': 'cylinder',
  'size': 'small'
  'behind': ['h1', 'h2', 'h3',...],
  'front': [ ],
  'left': ['h2', 'h3', ...],
  'right': ['h5', ...]},
 {'id:'h1',
  '3d_coords': [1.649, -1.450, 0.350],
  'color': 'yellow',
  'material': 'metal',
  'pixel_coords': [246, 201, 9.081],
  'rotation': 140.802,
  'shape': 'cube',
  'size': 'small'
  'behind': ['h2', 'h3', ...],
  'front': ['h0'],
  'left': ['h2', 'h3', ...],
  'right': ['h5', ...]},
]
```

Using CLEVR for H-data IV

- We can transform g into H-data h_s:

$$h_s = [\begin{bmatrix} \text{id} & = \text{a} \\ \text{data} = & \texttt{\{'id': 'h0, '3d\_coords':...\}} \end{bmatrix}, \begin{bmatrix} \text{id} & = \text{b} \\ \text{data} = & \texttt{\{'id': 'h1', '3d\_coords':...\}} \end{bmatrix}]$$

- From h_s and a background type T_{bg}, we can construct a witness take as described above

Using CLEVR for H-data V

- As mentioned above, we are using H-data as evidence/proofs of types, and judging if some H-data is of a ptype is done using classifiers.
- In the case where H-data are scene graphs, the classification becomes rather trivial as the required information is already available in the H-data.
- For example, a colour classifier might be specified as follows:

$$\kappa_{\text{col}} = \lambda h : \langle Hdata \rangle . \begin{cases} \text{RED if } h(1)\texttt{['color'] == 'red'} \\ \text{BLUE if } h(1)\texttt{['color'] == 'blue'} \\ \ldots \end{cases}$$

- For a two-place predicate:

$$\kappa_{\text{col}} = \lambda h : \langle Hdata, Hdata \rangle . \begin{cases} \text{LEFT if } h(2)\texttt{.['id'] in } h(1)\texttt{['left']} \\ \text{RIGHT if } h(2)\texttt{.['id'] in } h(1)\texttt{['right']} \end{cases}$$

Using CLEVR for H-data VI

- Note that we are here mixing TTR notation with `Python` code.
- This is specific to the type of H-data we are using and a particular way of accessing it.
- For other types of H-data, the classifier definition may look quite different.
- However, we have set things up so that the dependence on H-data type has been isolated to the classifier definitions.

Outline

Summary and future work I

- We outlined a formal account of VQA where an agent perceives a real world situation by
  1. receiving low-level perceptual data
  2. processing that data into high-level individuated perceptual data
  3. structuring this data in light of the semantics of a linguistic expression (a question)
  4. classifying the structured data to arrive at an answer to the question

Summary and future work II

- Importantly (although not directly in scope of this paper), word meanings in general, including perceptual meaning aspects modeled as classifiers, are dynamic and subject to learning and coordination between dialogue participants (Larsson, 2015)
- In future work, we aim to apply and extend the account to also encompass semantic pointers (Eliasmith, 2015), (Larsson *et al.*, 2023)

This work was supported by grants from the Swedish Research Council

- VR project 2025-04592, Semantic Pointers in Natural Language Semantics (SPiNLS)
- VR project 2014-39 for the establishment of the Centre for Linguistic Theory and Studies in Probability (CLASP) at the University of Gothenburg.

Bibliography I

Bekki, Daisuke; Tanaka, Ribeka; and Takahashi, Yuta 2021.
Integrating Deep Neural Network with Dependent Type Semantics.
In *Proceedings of the Symposium Logic and Algorithms in Computational Linguistics 2021 (LACompLing2021) 2021 (LACompLing2021)*, Stockholm University.
37.

Cooper, Robin 2023.
From Perception to Communication: a Theory of Types for Action and Meaning.
Oxford University Press.

Dobnik, Simon and Cooper, Robin 2017.
Interfacing language, spatial perception and cognition in type theory with records.
Journal of Language Modelling 5(2):273–301.

Bibliography II

Eliasmith, Chris 2015.
How to Build a Brain: A Neural Architecture for Biological Cognition.
Oxford Series on Cognitive Models and Architectures. Oxford Univ. Press.

Hudson, Drew A. and Manning, Christopher D. 2018.
Compositional Attention Networks for Machine Reasoning.
In *ICLR*. arXiv.
arXiv:1803.03067 [cs.AI].

Johnson, Justin; Hariharan, Bharath; Maaten, Laurens van der; Fei-Fei, Li; Zitnick, C. Lawrence; and Girshick, Ross 2016.
CLEVR: A Diagnostic Dataset for Compositional Language and Elementary Visual Reasoning.
arXiv:1612.06890 [cs].

Bibliography III

Johnson, Justin; Hariharan, Bharath; Van Der Maaten, Laurens; Hoffman, Judy; Fei-Fei, Li; Zitnick, C. Lawrence; and Girshick, Ross 2017.
Inferring and Executing Programs for Visual Reasoning.
In *2017 IEEE International Conference on Computer Vision (ICCV)*.
3008–3017.
ISSN: 2380-7504.

Larsson, Staffan; Cooper, Robin; Ginzburg, Jonathan; and Lücking, Andy 2023.
Ttr at the spa: Relating type-theoretical semantics to neural semantic pointers.
In *Proceedings of the 4th Natural Logic Meets Machine Learning Workshop*.
41–50.

Bibliography IV

Larsson, Staffan 2015.
Formal semantics for perceptual classification.
Journal of Logic and Computation 25(2):335–369.
Published online 2013-12-18.

Larsson, Staffan 2020.
Discrete and probabilistic classifier-based semantics.
In Howes, Christine; Chatzikyriakidis, Stergios; Ek, Adam; and Somashekarappa, Vidya, editors 2020, *Proceedings of the Probability and Meaning Conference (PaM 2020)*, Gothenburg. Association for Computational Linguistics.
62–68.

Bibliography V

Liu, Yinhong and Emerson, Guy 2022.
Learning Functional Distributional Semantics with Visual Data.
In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Dublin, Ireland. Association for Computational Linguistics.
3976–3988.

Matsson, Arild; Dobnik, Simon; and Larsson, Staffan 2019.
Imagettr: Grounding type theory with records in image classification for visual question answering.
In *Proceedings of the IWCS 2019 Workshop on Computing Semantics with Types, Frames and Related Structures*.
55–64.

Bibliography VI

Utescher, Ronja 2019.
Visual ttr-modelling visual question answering in type theory with records.
In *Proceedings of the 13th International Conference on Computational Semantics-Student Papers*.
9–14.

Yi, Kexin; Wu, Jiajun; Gan, Chuang; Torralba, Antonio; Kohli, Pushmeet; and Tenenbaum, Josh 2018.
Neural-Symbolic VQA: Disentangling Reasoning from Vision and Language Understanding.
In *Advances in Neural Information Processing Systems*, volume 31.
Curran Associates, Inc.